

DSA Day 36: Topological Sorting

Book: Data Structures Through C in Depth - Ch7 §7.15 · pp 410-416 | Code in Python

Overview

A topological order of a DAG lists each vertex before its dependents. Use in-degrees (Kahn) or DFS finish times.

Key concepts to master

- DAG
- in-degree
- ordering constraints

Python implementation

```
from collections import deque
def toposort(g,n):
    indeg=[0]*n
    for u in g:
        for v in g[u]: indeg[v]+=1
    q=deque([i for i in range(n) if indeg[i]==0]); out=[]
    while q:
        u=q.popleft(); out.append(u)
        for v in g[u]:
            indeg[v]-=1
            if indeg[v]==0: q.append(v)
    return out
```

Complexity

$O(V+E)$.

Common pitfalls

- A cycle makes topo sort impossible
- Not initialising in-degrees

Practice

- Order a 6-task DAG
- What breaks topological sort?

